SOFTWARE ARCHITECTURE DOCUMENT & REQUIREMENTS SPECIFICATION

FabricDefect AI

Intelligent Fabric Defect Detection System

Document Type	Software Architecture Document (SAD) & SRS
Domain	Industrial Inspection · Computer Vision · Generative AI
Team Structure	Team A — Synthetic Data · Team B — AI & Full Stack
Version	1.0
Status	Implementation-Ready

React + TypeScript | FastAPI | YOLO Segmentation | PostgreSQL | PyTorch | OpenCV

Table of Contents

1. Executive Summary

FabricDefect AI is an end-to-end intelligent inspection system that detects fabric defects using a YOLO segmentation model trained primarily on synthetic defect imagery. The system closes the synthetic-to-real data gap by pairing a dedicated **Synthetic Data Team (Team A)**, which generates and validates realistic defective fabric images using a fine-tuned generative model, with an **AI & Full Stack Development Team (Team B)**, which annotates the delivered dataset, trains and evaluates the segmentation model, and builds the complete production application around it.

The delivered product is a full-stack web platform — a FastAPI backend, a React/TypeScript frontend, and a PostgreSQL database — that allows textile quality-assurance staff to upload fabric images or run live camera inspection, view defect detections overlaid on the source image, review detection history, and monitor quality trends on an analytics dashboard. The system is authenticated, role-based, and exposes a REST API for all core operations.

This document is the authoritative technical reference for both teams. It specifies functional and non-functional requirements, the complete software and AI architecture, the API contract, database schema, folder structure, delivery roadmap, sprint plan, KPIs, testing strategy, and the UI/UX design system, so that implementation can proceed directly from this specification without further architectural decisions.

2. Problem Statement

Manual fabric inspection on production lines is slow, inconsistent, and dependent on the attentiveness of individual inspectors, which allows defective fabric (snags, holes, stains, weaving faults, broken yarns) to reach downstream processes or customers. Building an automated visual inspection model is difficult because real, labeled defect images are scarce, expensive to collect, and narrow in variety — a single factory may only produce a handful of examples of a rare defect type per month.

Synthetic image generation can multiply the available training data, but a model trained purely on synthetic images typically under-performs when deployed against real camera feeds because of differences in lighting, fabric texture, weave micro-structure, and camera noise between synthetic and real domains. **FabricDefect AI addresses this by treating synthetic data generation as a first-class, validated pipeline stage (owned by Team A)** feeding into a rigorous annotation and training pipeline (owned by Team B), rather than treating synthetic data as an ad-hoc shortcut.

3. Project Objectives

- Build a repeatable pipeline that fine-tunes a generative model on clean fabric images and produces validated, realistic synthetic defective fabric images.
- Annotate the synthetic dataset for instance/semantic segmentation using CVAT or Roboflow and prepare it in YOLO segmentation format.
- Train, validate, and evaluate a YOLO segmentation model dedicated to fabric defect localization and classification.
- Package the trained model behind a reusable AI inference module callable by the backend.
- Design and implement a secure, role-based FastAPI backend exposing authentication, image upload, live camera detection, detection history, and analytics endpoints.
- Design and implement a responsive React/TypeScript frontend covering authentication, upload/live-detection workflows, history browsing, and a KPI dashboard.
- Establish a database schema that cleanly separates users, uploaded images, detection results, and defect metadata.
- Define a joint delivery roadmap, sprint plan, and KPI framework so that Team A and Team B can work in parallel with clear, synchronized handoff points.

4. Project Scope

In Scope

- Fabric defect detection exclusively (no other inspection domains).
- Synthetic dataset generation, validation, and metadata delivery.
- YOLO segmentation dataset preparation, training, and evaluation.
- AI inference module for image-based and live-camera detection.
- FastAPI backend with authentication, upload, live detection, history, analytics, and REST API.
- React frontend for all end-user workflows and dashboards.
- PostgreSQL schema for users, images, detections, and roles.

Out of Scope

- Non-fabric defect categories (PCB, metal, plastics, etc.).
- Deployment architecture, containerization, cloud infrastructure, CI/CD, or monitoring.
- ER diagrams, UML/sequence/activity/component/data-flow diagrams, and wireframes.
- Risk analysis and future-scope planning.
- Hardware selection for cameras or lighting rigs.
- Multi-tenant / multi-factory federation (single-deployment scope only).

5. Functional Requirements

ID	Requirement	Description
FR-01	User Authentication	Users register/login via email + password; the system issues a JWT access token and refresh token.
FR-02	Role-Based Access	The system enforces permissions for Admin, QA Inspector, and Viewer roles on every protected endpoint.
FR-03	Image Upload	Users upload one or more fabric images (JPEG/PNG) for defect detection via the web UI.
FR-04	Live Camera Detection	Users stream a connected camera feed into the browser; frames are sent to the backend for near-real-time inference.
FR-05	Defect Detection	The AI inference module returns segmentation masks, defect class, confidence score, and bounding region for each detected defect.
FR-06	Detection Overlay	The frontend renders detected defect masks/boxes over the source image or live frame.
FR-07	Detection History	Every detection event (image, results, timestamp, user) is persisted and browsable/filterable.
FR-08	Dashboard & Analytics	The system aggregates detection counts, defect-type distribution, and confidence trends into dashboard charts.
FR-09	Synthetic Dataset Intake	The AI pipeline ingests Team A's synthetic dataset package (images + generation metadata) as training input.
FR-10	Annotation Workflow	Synthetic images are annotated in CVAT/Roboflow and exported in YOLO segmentation format.
FR-11	Model Training & Evaluation	The AI module trains a YOLO segmentation model and produces evaluation metrics (precision, recall, mAP, IoU).
FR-12	REST API	All core operations (auth, upload, detect, history, analytics) are exposed through a documented REST API.
FR-13	User Management	Admins can create, update, deactivate, and assign roles to user accounts.

6. Non-Functional Requirements

Category	Requirement
Performance	Single-image inference completes in under 1.5 seconds on the target inference hardware; live-camera mode sustains at least 8 processed frames per second.
Scalability	Backend services are stateless behind the REST API layer so that additional API instances can be added without code changes.
Security	All endpoints (except login/register) require a valid JWT; passwords are stored using salted hashing (bcrypt/argon2); role checks are enforced server-side.
Reliability	Failed inference requests return structured error responses and do not corrupt detection history records.
Usability	The frontend follows the defined UI/UX design system (Section 23) for consistent, accessible interaction.
Maintainability	Backend, frontend, and AI module are decoupled through clearly defined interfaces (Section 16) so each can evolve independently.
Data Integrity	All detection records reference immutable image storage identifiers; database relationships enforce referential integrity via foreign keys.
Auditability	Every detection and user-management action is timestamped and attributable to a specific user account.

7. User Roles and Permissions

Role	Description	Permissions
Admin	Manages the platform and its users.	Full access: user management, all detection features, dashboard, model/version metadata viewing, system configuration.
QA Inspector	Performs day-to-day fabric inspection.	Upload images, run live camera detection, view own and team detection history, view dashboard.
Viewer	Reviews outcomes without operating the inspection tools.	Read-only access to detection history and dashboard analytics.

8. User Stories

ID	Role	Story
US-01	QA Inspector	As a QA Inspector, I want to upload a fabric image and instantly see any detected defects highlighted, so I can decide whether to reject the roll.
US-02	QA Inspector	As a QA Inspector, I want to run live camera detection at my inspection station, so I can catch defects as fabric passes by without manual uploads.
US-03	QA Inspector	As a QA Inspector, I want to browse my past detections by date and defect type, so I can review recurring issues from a specific shift.
US-04	Admin	As an Admin, I want to create accounts and assign roles, so only authorized staff can operate detection tools.
US-05	Admin	As an Admin, I want a dashboard showing defect-type distribution and volume trends, so I can report quality metrics to management.
US-06	Viewer	As a Viewer (e.g., a plant manager), I want read-only access to the dashboard and history, so I can monitor quality without operating inspection equipment.
US-07	Team A Member	As a synthetic data engineer, I want to package validated synthetic images with generation metadata, so Team B can ingest a trustworthy, versioned dataset.
US-08	Team B AI Engineer	As an AI engineer, I want reproducible training/evaluation runs with logged metrics, so I can compare model versions before promoting one to production.

9. Use Cases

Use Case	Actor	Main Flow	Outcome
UC-01 Upload & Detect	QA Inspector	Login → open Upload page → select image(s) → submit → backend calls AI inference module → results returned	Defects, classes, and confidence scores displayed overlaid on the image; record saved to history.
UC-02 Live Detection	QA Inspector	Login → open Live Detection page → grant camera access → stream frames to backend at fixed interval → receive per-frame results	Real-time overlay on the live feed; significant detections logged to history.
UC-03 Review History	QA Inspector / Viewer / Admin	Open History page → filter by date/user/defect type → select a record → view detail	Full detail view of a past detection including original image and result metadata.
UC-04 View Dashboard	Admin / Viewer	Open Dashboard → system aggregates detection data over the selected period	Charts for defect distribution, volume trend, and average confidence are rendered.
UC-05 Manage Users	Admin	Open Users page → create/edit/deactivate account → assign role	User record updated; permissions take effect on next request.
UC-06 Generate Synthetic Dataset	Team A Member	Collect clean images → fine-tune generative model → generate defect images → validate → export package	Versioned synthetic dataset with metadata delivered to Team B's ingestion folder.
UC-07 Train Model	Team B AI Engineer	Ingest dataset → annotate → prepare YOLO dataset → train → evaluate	Trained model weights and evaluation report produced; best model promoted to the inference module.

10. Complete Software Architecture

FabricDefect AI follows a **layered, service-oriented architecture** with four cooperating subsystems: the synthetic data subsystem (Team A), the AI subsystem (dataset preparation, training, inference — Team B/AI), the backend service (FastAPI), and the frontend application (React). The backend is the single integration point: it owns the database, exposes the REST API, and is the only component permitted to call the AI inference module directly. The frontend never talks to the AI module or database directly — all access is mediated through the REST API layer.

Layer	Responsibility	Key Technologies
Presentation Layer	User interaction, image capture, result visualization, dashboards	React, Vite, TypeScript, Tailwind CSS, Shadcn UI, React Router, Axios
API Layer	Request routing, validation, authentication/authorization, response shaping	FastAPI, Pydantic, JWT
Service Layer	Business logic: user management, upload handling, history aggregation, analytics	Python service modules (FastAPI routers + services)
AI Inference Layer	Model loading, preprocessing, segmentation inference, postprocessing	PyTorch, YOLO segmentation, OpenCV
Data Layer	Persistent storage of users, images metadata, detections, roles	PostgreSQL, SQLAlchemy
Synthetic Data Layer	Clean image ingestion, generative fine-tuning, synthetic generation, validation	Generative AI model, image-quality validation scripts

> The full end-to-end diagram combining all layers, both team workflows, and user interaction is provided in **Section 24**.

11. AI Pipeline Architecture

The AI pipeline is a linear, checkpointed sequence owned jointly by Team A (first stage) and Team B (remaining stages). Each stage produces a versioned artifact consumed by the next stage.

Stage	Stage Name	Input → Output	Owner
1	Clean Image Collection	Raw fabric photos → curated clean-image set	Team A
2	Generative Fine-Tuning	Clean-image set → fine-tuned generative model checkpoint	Team A
3	Synthetic Generation	Fine-tuned model → raw synthetic defect images	Team A
4	Quality Validation	Raw synthetic images → validated synthetic dataset + rejection log	Team A
5	Dataset Ingestion	Validated dataset package → local annotation workspace	Team B
6	Annotation	Workspace images → CVAT/Roboflow segmentation annotations	Team B
7	YOLO Dataset Prep	Annotations → YOLO-format train/val/test splits	Team B
8	Model Training	YOLO dataset → trained segmentation weights	Team B
9	Model Evaluation	Trained weights + val/test split → metrics report	Team B
10	AI Inference Module	Promoted weights → callable inference service used by backend	Team B

Inference Module Responsibilities

- Load the promoted YOLO segmentation model once at service start-up.
- Preprocess incoming images/frames (resize, normalize, color-space conversion via OpenCV).
- Run segmentation inference and post-process masks into structured defect objects (class, confidence, polygon/box, area).
- Return a normalized JSON-serializable result to the calling backend service.

12. Team Collaboration Workflow

Team A and Team B work in parallel with a single synchronized handoff: the validated synthetic dataset package. Team B does not wait for Team A to fully finish before starting backend/frontend development, since those tracks are independent of the dataset until the AI inference module is ready for integration.

Milestone	Team A Activity	Team B Activity
Kickoff	Define fabric defect taxonomy, begin clean-image collection	Set up repositories, scaffold backend/frontend, define DB schema
Mid-Cycle 1	Fine-tune generative model, generate first synthetic batch	Build auth, user management, upload endpoints and UI
Handoff Point	Deliver validated synthetic dataset + metadata (Section 21, Team A KPIs)	Ingest dataset, begin annotation in CVAT/Roboflow
Mid-Cycle 2	Iterate on generation quality based on annotation/training feedback	Prepare YOLO dataset, train and evaluate model
Integration	Deliver refined dataset versions as needed	Wire AI inference module into backend; build live detection UI
Stabilization	Finalize dataset documentation	End-to-end testing, dashboard/analytics polish, documentation

> Communication contract: Team A publishes every dataset delivery with a semantic version (e.g., `v1.2`), an image count, defect-class breakdown, and a generation-metadata file so Team B's training runs are fully reproducible and traceable to a specific dataset version.

13. Frontend Architecture

The frontend is a single-page application built with React, Vite, and TypeScript, styled with Tailwind CSS and Shadcn UI components, and routed with React Router. All server communication goes through a centralized Axios client with interceptors for JWT attachment and refresh handling.

Module	Responsibility
Auth Module	Login, registration, token storage/refresh, protected-route guards
Upload & Detection Module	Image selection/upload, submission to detection endpoint, result overlay rendering
Live Camera Module	Browser camera access, frame capture loop, streaming to backend, real-time overlay
History Module	Paginated/filterable detection list, detail view
Dashboard Module	Chart rendering for defect distribution, trends, and confidence statistics
User Management Module	(Admin only) account creation, role assignment, deactivation
Shared UI Kit	Shadcn UI-based buttons, forms, tables, modals, navigation shell used by all modules
API Client Layer	Axios instance, request/response typing, error normalization

State & Data Flow

Server state (detections, history, dashboard aggregates) is fetched per-view through the API client layer and cached at the component level; there is no direct AI-module or database access from the frontend. Authentication state (current user, role, token) is held in a top-level context provider and consumed by route guards and the navigation shell.

14. Backend Architecture

The backend is a FastAPI application organized into routers, services, and repositories. Routers handle HTTP concerns and validation (via Pydantic schemas); services implement business logic; repositories encapsulate SQLAlchemy database access. The backend is the only component that invokes the AI inference module.

Component	Responsibility
Auth Router/Service	Registration, login, JWT issuance/refresh, password hashing
User Router/Service	CRUD for user accounts, role assignment (Admin only)
Upload Router/Service	Receive image files, store them, invoke AI inference, persist detection results
Live Detection Router/Service	Receive streamed frames, invoke AI inference synchronously per frame, return results
History Router/Service	Query and filter past detections; return paginated results
Analytics Router/Service	Aggregate detection data into dashboard-ready summaries
AI Client Module	Thin wrapper that calls the AI inference module and normalizes its output
Core/Config	Settings, JWT configuration, database session management, dependency injection

Cross-Cutting Concerns

- **Authentication:** JWT bearer tokens validated on every protected route via a FastAPI dependency.
- **Authorization:** Role checks implemented as reusable FastAPI dependencies (e.g., `require_role("Admin")`).
- **Validation:** All request/response bodies defined as Pydantic models.
- **Error Handling:** Centralized exception handlers return a consistent error response schema.

15. AI Module Architecture

The AI module is a Python package independent of the web framework, so it can be trained, evaluated, and versioned separately from the backend, then imported as a library by the backend's AI Client Module.

Sub-Module	Responsibility
`data_prep`	Converts CVAT/Roboflow exports into YOLO segmentation folder structure (images/labels, train/val/test splits)
`training`	Configures and runs YOLO segmentation training (hyperparameters, augmentation, checkpointing)
`evaluation`	Computes precision, recall, mAP@0.5, mAP@0.5:0.95, and per-class IoU on validation/test splits
`inference`	Loads a promoted model checkpoint and exposes a single `predict(image) → DetectionResult` function
`preprocessing`	OpenCV-based resize/normalize/color-conversion shared by training and inference
`schemas`	Typed result objects (defect class, confidence, mask/box, area) shared with the backend

> The `inference` sub-module is the only part of the AI Module imported by the backend at runtime; `training` and `data_prep` run offline as part of the model development cycle.

16. API Structure

Method	Endpoint	Auth	Description
POST	`/api/v1/auth/register`	Public	Create a new user account
POST	`/api/v1/auth/login`	Public	Authenticate and receive JWT access/refresh tokens
POST	`/api/v1/auth/refresh`	Refresh token	Issue a new access token
GET	`/api/v1/users/me`	Any role	Get current user profile
GET	`/api/v1/users`	Admin	List all users
POST	`/api/v1/users`	Admin	Create a user account
PATCH	`/api/v1/users/{id}`	Admin	Update role/status of a user
POST	`/api/v1/detections/upload`	Inspector, Admin	Upload image(s) and run defect detection
POST	`/api/v1/detections/live-frame`	Inspector, Admin	Submit a single live-camera frame for detection
GET	`/api/v1/detections`	Any role	List/filter detection history (paginated)
GET	`/api/v1/detections/{id}`	Any role	Get full detail of one detection record
GET	`/api/v1/analytics/summary`	Any role	Aggregate KPIs for the dashboard (counts, distribution, trends)
GET	`/api/v1/analytics/defect-distribution`	Any role	Defect-type breakdown for the selected period

17. Database Design

Tables and relationships only, as specified — no ER diagram.

Table	Key Columns	Relationships
users	id (PK), full_name, email (unique), password_hash, role, is_active, created_at	1 → many **detections** (uploaded_by)
roles	id (PK), name (Admin / QA Inspector / Viewer)	1 → many **users** (role_id)
images	id (PK), file_path, source_type (upload / live_frame), uploaded_by (FK → users.id), created_at	1 → 1 **detections** (image_id)
detections	id (PK), image_id (FK → images.id), user_id (FK → users.id), model_version, overall_confidence, created_at	1 → many **defect_results**
defect_results	id (PK), detection_id (FK → detections.id), defect_class, confidence, mask_data, area	many → 1 **detections**; many → 1 **defect_classes**
defect_classes	id (PK), name, description	1 → many **defect_results**
model_versions	id (PK), version_tag, dataset_version, trained_at, metrics_json	1 → many **detections** (model_version reference)
refresh_tokens	id (PK), user_id (FK → users.id), token_hash, expires_at, revoked	many → 1 **users**

18. Complete Folder Structure

Frontend

```
frontend/
├── src/
│   ├── assets/
│   ├── components/
│   │   ├── ui/                # Shadcn UI primitives (button, card, dialog, table...)
│   │   ├── layout/             # Navbar, Sidebar, AppShell
│   │   └── detection/          # DetectionOverlay, DefectBadge, ConfidenceBar
│   ├── features/
│   │   ├── auth/                # LoginPage, RegisterPage, authSlice, useAuth
│   │   ├── upload/              # UploadPage, UploadForm, ResultView
│   │   ├── live-detection/      # LiveDetectionPage, CameraStream, LiveOverlay
│   │   ├── history/             # HistoryPage, HistoryTable, DetectionDetail
│   │   ├── dashboard/           # DashboardPage, charts/
│   │   └── users/               # UsersPage, UserForm  (Admin only)
│   ├── lib/
│   │   ├── apiClient.ts         # Axios instance + interceptors
│   │   ├── types.ts             # Shared TS types/interfaces
│   │   └── utils.ts
│   ├── routes/
│   │   ├── AppRouter.tsx
│   │   └── ProtectedRoute.tsx
│   ├── styles/
│   │   └── globals.css          # Tailwind base + design tokens
│   ├── App.tsx
│   └── main.tsx
├── public/
├── index.html
├── tailwind.config.ts
├── tsconfig.json
├── vite.config.ts
└── package.json
```

Backend

```
backend/
├── app/
│   ├── api/
│   │   └── v1/
│   │       ├── auth.py
│   │       ├── users.py
│   │       ├── detections.py
│   │       └── analytics.py
│   ├── core/
│   │   ├── config.py
│   │   ├── security.py          # JWT + password hashing
│   │   └── dependencies.py      # get_current_user, require_role
│   ├── models/
│   │   ├── user.py
│   │   ├── image.py
│   │   ├── detection.py
│   │   ├── defect_result.py
│   │   └── model_version.py
│   ├── schemas/
│   │   ├── auth.py
│   │   ├── user.py
│   │   ├── detection.py
│   │   └── analytics.py
│   ├── services/
│   │   ├── auth_service.py
│   │   ├── user_service.py
│   │   ├── detection_service.py
│   │   └── analytics_service.py
│   ├── ai_client/
│   │   └── inference_client.py   # wraps ai_module.inference
│   ├── db/
│   │   ├── base.py
│   │   ├── session.py
│   │   └── repository.py
│   └── main.py
├── tests/
│   ├── test_auth.py
│   ├── test_detections.py
│   └── test_analytics.py
├── alembic/                      # DB migrations
├── requirements.txt
└── pyproject.toml
```

AI Module

```
ai_module/
├── data_prep/
│   ├── cvat_to_yolo.py
│   ├── roboflow_export.py
│   └── split_dataset.py
├── training/
│   ├── train_yolo_seg.py
│   ├── config/
│   │   └── fabric_defect.yaml
│   └── augmentations.py
├── evaluation/
│   ├── evaluate.py
│   └── metrics.py
├── inference/
│   ├── predictor.py              # predict(image) -> DetectionResult
│   └── model_loader.py
├── preprocessing/
│   └── image_ops.py               # OpenCV resize/normalize
├── schemas/
│   └── detection_result.py
├── weights/
│   └── fabric_yolo_seg_vX.pt
└── requirements.txt
```

Shared Resources

```
shared/
├── synthetic_dataset/
│   ├── raw/                       # Team A generated images
│   ├── validated/                 # Passed quality validation
│   ├── metadata/
│   │   └── generation_metadata.json
│   └── VERSION
├── annotations/
│   └── yolo_seg/                  # CVAT/Roboflow → YOLO export, consumed by ai_module
├── docs/
│   ├── api_reference.md
│   ├── dataset_changelog.md
│   └── defect_taxonomy.md
└── scripts/
    └── dataset_handoff_check.py     # validates a Team A delivery package
```

19. Development Roadmap

Phase	Focus	Key Outputs
Phase 1	Foundation	Repo scaffolding, DB schema migration, auth flow, defect taxonomy defined, clean-image collection started
Phase 2	Data Generation	Generative model fine-tuned, first synthetic batches produced and validated
Phase 3	Core Backend/Frontend	Upload flow, user management, history module, dashboard skeleton
Phase 4	Dataset Handoff & Annotation	Validated dataset delivered; CVAT/Roboflow annotation completed; YOLO dataset prepared
Phase 5	Model Training	YOLO segmentation model trained, evaluated, and promoted; AI inference module finalized
Phase 6	Integration	Backend wired to AI inference module; live camera detection implemented end-to-end
Phase 7	Analytics & Polish	Dashboard analytics completed, UI/UX design system fully applied
Phase 8	Stabilization	End-to-end testing, bug fixing, documentation finalized

20. Sprint Planning

Sprint	Team A	Team B
Sprint 1	Define defect taxonomy; begin clean-fabric image collection	Scaffold backend/frontend repos; DB schema + migrations; auth endpoints
Sprint 2	Continue image collection; prepare fine-tuning dataset	User management UI/API; upload endpoint (stub inference)
Sprint 3	Fine-tune generative model; first synthetic generation run	Upload UI + result overlay component; history module (backend + UI)
Sprint 4	Quality validation pipeline; package dataset v1.0	Dashboard skeleton; analytics endpoints (mock data)
Sprint 5	**Deliver dataset v1.0 to Team B**; begin v1.1 improvements	Ingest dataset; CVAT/Roboflow annotation begins
Sprint 6	Iterate generation quality from annotation feedback	Complete annotation; YOLO dataset preparation; begin training
Sprint 7	Deliver dataset v1.1 (expanded/edge cases)	Model training + evaluation cycles
Sprint 8	Dataset documentation finalized	Promote best model; build AI inference module
Sprint 9	Support any final data gaps identified during integration	Integrate inference module into backend; live camera detection
Sprint 10	—	Dashboard analytics polish; UI/UX design system pass
Sprint 11	—	End-to-end testing, bug fixing
Sprint 12	—	Documentation, final stabilization

21. KPI Plan

Team A KPIs

KPI	Target
Synthetic image validation pass rate	≥ 85% of generated images pass quality validation
Dataset delivery cadence	One versioned dataset delivery per sprint during Sprints 5–8
Defect-class coverage	All defect classes in the taxonomy represented in every delivered dataset version
Generation metadata completeness	100% of delivered images accompanied by complete generation metadata
Visual realism score	Improvement trend across dataset versions, tracked via validation review notes

Team B KPIs

KPI	Target
Model mAP@0.5 (segmentation)	≥ 0.80 on held-out real/validation fabric images
Inference latency (single image)	< 1.5 seconds end-to-end
Live detection frame rate	≥ 8 FPS sustained
API test coverage	≥ 80% coverage on service-layer code
Sprint delivery predictability	≥ 90% of committed sprint items completed on time
Critical defect escape rate (post-integration testing)	0 critical defects reaching stabilization phase

22. Testing Strategy

Level	Scope	Approach
Unit Testing	Backend services, AI module functions, React components/hooks	Pytest for backend/AI module; Vitest + React Testing Library for frontend
Integration Testing	API routes against a test database; AI inference client against a fixed model checkpoint	Pytest with a dedicated test PostgreSQL schema and fixture data
Model Evaluation Testing	Segmentation quality on held-out validation/test splits	Precision, recall, mAP@0.5, mAP@0.5:0.95, per-class IoU thresholds gating model promotion
End-to-End Testing	Full upload → detect → history → dashboard flow, and live-camera flow	Scripted E2E flows exercising the real backend and a test build of the frontend
Security Testing	Auth boundaries, role enforcement, token expiry/refresh	Negative test cases per endpoint for unauthorized and cross-role access
Regression Testing	Re-run full suite on every model promotion or schema change	Automated test suite executed before merging to the main branch

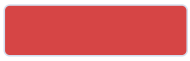
23. UI/UX Design System

Color Palette


Primary — #1F3A63


Primary Accent — #2A6FDB


Success / Pass — #1F9D6B


Defect / Alert — #D64545


Warning — #E0A32E

Surface / Background — #F4F6FB


Text Primary — #1C2333


Text Secondary — #667085

Typography

Role	Typeface	Usage
Headings	Inter / system sans-serif, Semi-Bold	Page titles, section headers, dashboard KPI labels
Body	Inter / system sans-serif, Regular	Paragraphs, table content, form labels
Monospace	JetBrains Mono / system monospace	Model version tags, IDs, metadata values

Component Style

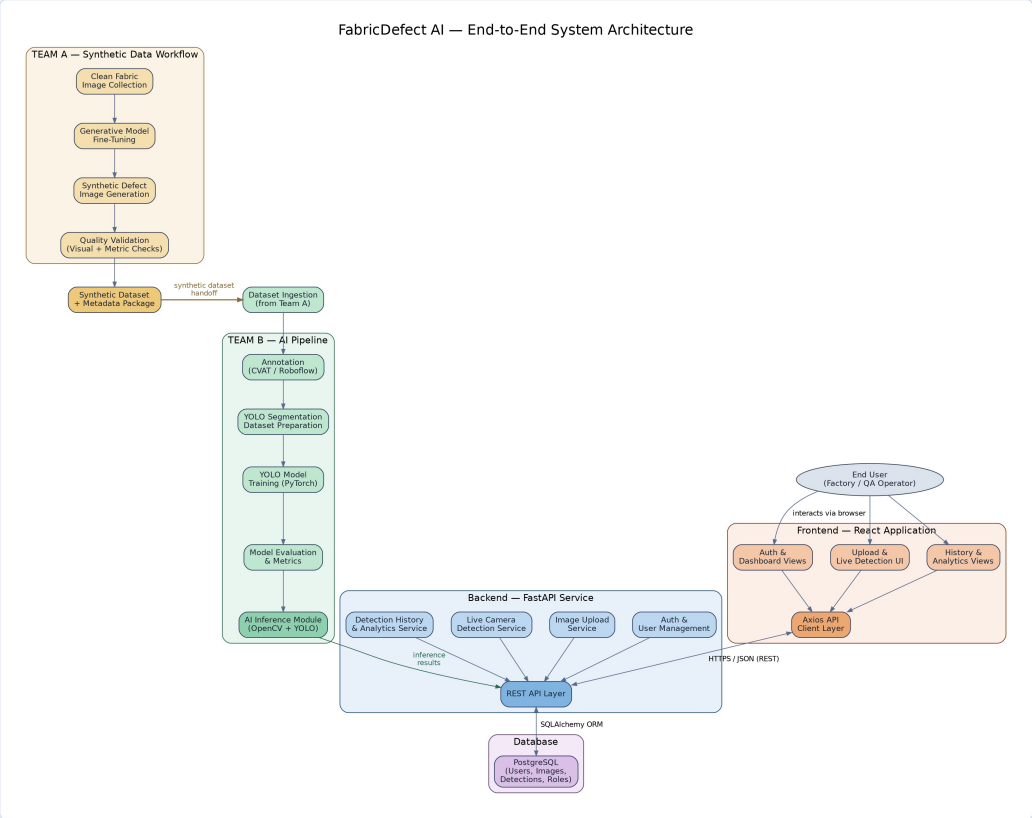
- Built on **Shadcn UI** primitives with Tailwind utility classes; no custom component library maintained separately.
- Cards with 8px corner radius and a subtle 1px border used for all content containers (upload panel, history rows, dashboard tiles).
- Primary actions use solid accent-color buttons; secondary/destructive actions use outline and red-toned variants respectively.
- Defect detections are always color-coded consistently: red-family for defect overlays, green for "pass"/no-defect states.

Dashboard Theme

- Light, high-contrast surface (`#F4F6FB` background, white cards) to keep focus on defect imagery and charts.
- KPI tiles at the top (total detections, defect rate, average confidence, top defect class) followed by a defect-distribution chart and a volume-over-time trend chart.
- Consistent iconography for defect classes across the dashboard, history table, and detection overlay legend.

24. End-to-End System Architecture Diagram

The diagram below consolidates the Team A synthetic-data workflow, the Team B AI pipeline, the FastAPI backend, the PostgreSQL database, the React frontend, and end-user interaction into a single system view.



Legend: gold nodes — Team A synthetic data workflow; green nodes — Team B AI pipeline; blue nodes — FastAPI backend services; purple node — PostgreSQL database; orange nodes — React frontend; grey ellipse — end user. Arrows indicate data flow direction, including the synthetic dataset handoff from Team A to Team B and the bidirectional REST API traffic between frontend and backend.